

# Weeks 1–2: From Biological Neurons to the Perceptron

Deep Learning – IITM BS  
Notes with Intuition + Diagrams

## Contents

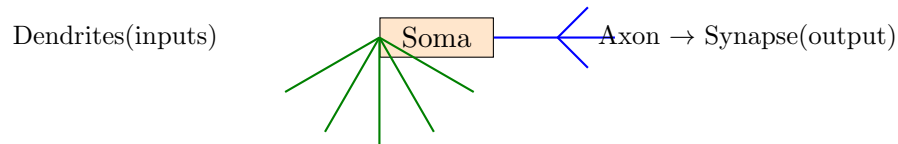
|          |                                                           |          |
|----------|-----------------------------------------------------------|----------|
| <b>1</b> | <b>The Big Picture: Where Are We Going?</b>               | <b>2</b> |
| 1.1      | Biological Inspiration . . . . .                          | 2        |
| <b>2</b> | <b>The McCulloch-Pitts (MP) Neuron</b>                    | <b>2</b> |
| 2.1      | Definition . . . . .                                      | 2        |
| 2.2      | MP Neuron Implements Boolean Functions . . . . .          | 3        |
| 2.3      | Linear Separability . . . . .                             | 3        |
| <b>3</b> | <b>The Perceptron</b>                                     | <b>4</b> |
| 3.1      | From MP Neuron to Perceptron: What Changed? . . . . .     | 4        |
| 3.2      | Geometric View: The Decision Boundary . . . . .           | 4        |
| <b>4</b> | <b>Perceptron Learning Algorithm</b>                      | <b>5</b> |
| 4.1      | The Core Idea . . . . .                                   | 5        |
| 4.2      | Convergence Guarantee . . . . .                           | 6        |
| 4.3      | Linearly Separable Sets: Formal Definition . . . . .      | 6        |
| <b>5</b> | <b>Representation Power: One Perceptron is Not Enough</b> | <b>6</b> |
| 5.1      | The XOR Problem . . . . .                                 | 6        |
| 5.2      | The Fix: Networks of Perceptrons . . . . .                | 6        |
| <b>6</b> | <b>Practice Problems</b>                                  | <b>8</b> |
| <b>7</b> | <b>Solutions</b>                                          | <b>9</b> |

# 1 The Big Picture: Where Are We Going?

## Intuition:

Why start with biology? Every deep learning model, no matter how complex (Transformers, CNNs, LSTMs), is built from the same tiny Lego brick: a single artificial neuron that takes some numbers in, combines them, and decides an output. Understanding the MP Neuron and Perceptron deeply means you understand *every* deep network at its most fundamental level—they are just millions of these tiny units connected together.

## 1.1 Biological Inspiration



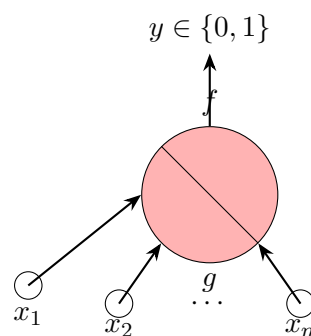
- **Dendrite:** receives signals from other neurons (= **inputs**  $x_1, \dots, x_n$ )
- **Soma:** processes/aggregates the information (= **summation function**  $g$ )
- **Axon:** transmits the output (= **output**  $y$ )
- **Synapse:** connection point to next neuron (= **weight** on the connection)

## Intuition:

Historical arc (good for MCQs) Reticular Theory (one continuous network) → Neuron Doctrine (network of discrete cells, the correct view) → McCulloch-Pitts Neuron (1943, first math model) → Perceptron (Rosenblatt, 1958, learnable weights) → Minsky & Papert's book *Perceptrons* (1969, showed limitations like XOR) → AI Winter → Backpropagation revives the field → Universal Approximation Theorem.

# 2 The McCulloch-Pitts (MP) Neuron

## 2.1 Definition



**MP Neuron Equations**

Inputs  $x_1, \dots, x_n \in \{0, 1\}$  (boolean), some may be **inhibitory**.

**Aggregation:**  $g(x_1, \dots, x_n) = \sum_i x_i$  (only over excitatory inputs)

**Decision:**

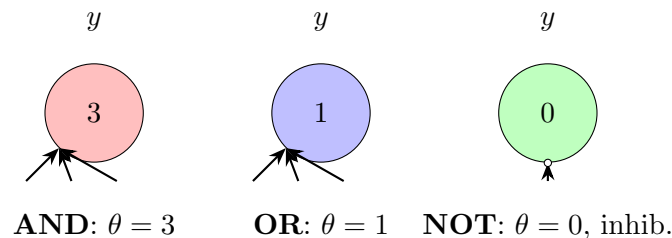
$$y = f(g(x)) = \begin{cases} 1 & \text{if } g(x) \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

$\theta$  = **thresholding parameter**.

**Inhibitory rule:** If *any* inhibitory input is 1, output is **forced to 0**, regardless of everything else.

**Intuition:**

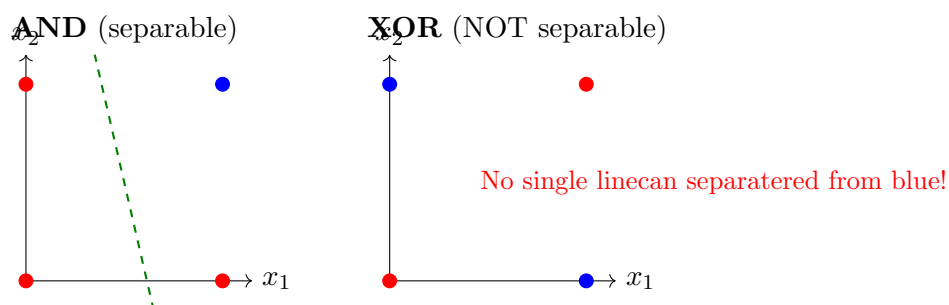
What does "inhibitory" really mean? Think of it as a veto power. Excitatory inputs *vote* for firing (add to the sum), but a single inhibitory input can *override* all the votes. Biologically: certain neurotransmitters actively suppress firing no matter how excited the rest of the cell is.

**2.2 MP Neuron Implements Boolean Functions**

- **AND** ( $n$  inputs):  $\theta = n$  (need all inputs on)
- **OR** ( $n$  inputs):  $\theta = 1$  (need at least one)
- **NOT:** single **inhibitory** input,  $\theta = 0$  (fires when input is 0)
- **Tautology** (always ON):  $\theta = 0$

**2.3 Linear Separability****Intuition:**

What is linear separability, visually? Plot all input combinations as points in space (for 2 inputs, a 2D plane). If you can draw a single straight line (or plane, in higher dimensions) that puts all the "1" outputs on one side and all the "0" outputs on the other, the function is **linearly separable**. An MP neuron (and a single perceptron) can *only* represent such functions.



**Key Counting Facts**

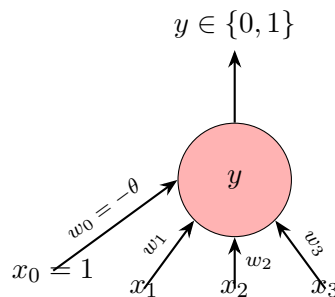
- Number of boolean functions of  $n$  inputs:  $2^{2^n}$
- For  $n = 2$ :  $2^4 = 16$  possible functions; only **XOR** and **NXOR** are NOT linearly separable
- A single MP neuron / perceptron can represent **any** linearly separable boolean function

### 3 The Perceptron

#### 3.1 From MP Neuron to Perceptron: What Changed?

**Intuition:**

The key upgrade The MP neuron's weights are all fixed at 1 (no learning) and inputs must be boolean. The Perceptron's big idea: give every input a **numerical, learnable weight**, and allow **real-valued** inputs. Now the machine can *learn* the right weights from data instead of a human hard-coding  $\theta$ .

**Perceptron Equation**

$$y = 1 \text{ if } \sum_{i=1}^n w_i x_i \geq \theta, \text{ else } 0$$

**Bias trick:** let  $w_0 = -\theta$  and  $x_0 = 1$  (always). Then:

$$y = 1 \text{ if } \sum_{i=0}^n w_i x_i \geq 0, \text{ else } 0$$

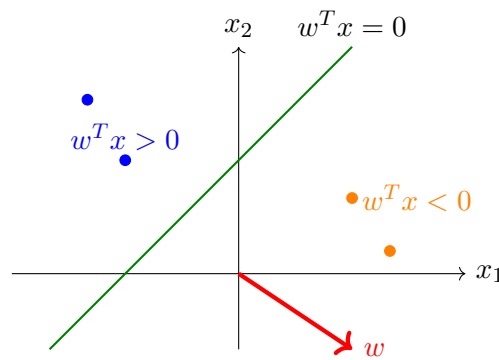
Or compactly using dot product:  $y = 1$  if  $w^T x \geq 0$ , else 0.

$w_0$  is called the **bias** – represents the “prior” or built-in prejudice of the neuron (how easy/hard it is to fire regardless of input).

#### 3.2 Geometric View: The Decision Boundary

**Intuition:**

Why is  $w$  perpendicular to the decision line? The equation  $w^T x = 0$  defines a line (or hyperplane) through the origin. For any point  $x$  on this line,  $\cos \alpha = \frac{w^T x}{\|w\| \|x\|} = 0$ , meaning the angle between  $w$  and  $x$  is exactly 90. Since this holds for *every* point on the line,  $w$  must be perpendicular to the entire line. This single fact is the geometric key to the whole Perceptron algorithm:  $w$  is like an arrow pointing in the direction of “positive class”.



- Points where  $w^T x > 0$ : angle with  $w$  is  $< 90^\circ \rightarrow$  classified as 1
- Points where  $w^T x < 0$ : angle with  $w$  is  $> 90^\circ \rightarrow$  classified as 0
- The perceptron finds a hyperplane that splits the space into two halves

## 4 Perceptron Learning Algorithm

### 4.1 The Core Idea

#### *Intuition:*

Learning as "nudging" the weight vector. Every time the perceptron misclassifies a point, we *nudge*  $w$  a little bit in a direction that would have helped classify that point correctly. Do this over and over across the dataset, and (if the data allows it)  $w$  eventually settles into a position that classifies everything correctly.

#### The Algorithm

Let  $P$  = points with label 1,  $N$  = points with label 0.

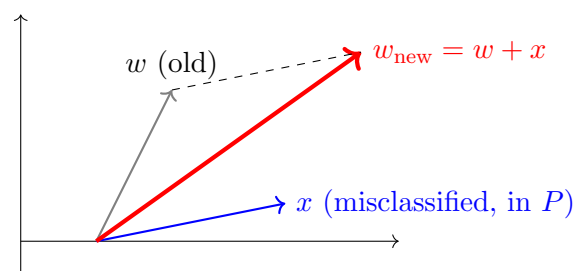
1. Initialize  $w$  randomly
2. Repeat until convergence:
  - Pick random  $x \in P \cup N$
  - If  $x \in P$  and  $w^T x < 0$  (misclassified positive):  $w \leftarrow w + x$
  - If  $x \in N$  and  $w^T x \geq 0$  (misclassified negative):  $w \leftarrow w - x$

#### *Intuition:*

Why does adding  $x$  to  $w$  help? If  $x \in P$  was misclassified, the angle between  $w$  and  $x$  was  $> 90^\circ$  (too far). After the update  $w_{\text{new}} = w + x$ :

$$w_{\text{new}}^T x = (w + x)^T x = w^T x + \|x\|^2$$

Since  $\|x\|^2 > 0$ , the new dot product is *larger* than before—moving the angle *closer* to  $90^\circ$  or below. Repeating this consistently pulls  $w$  toward the correct orientation. (Symmetric argument for  $x \in N$  with subtraction.)



## 4.2 Convergence Guarantee

### Perceptron Convergence Theorem

If the sets  $P$  and  $N$  are **linearly separable**, the Perceptron Learning Algorithm is guaranteed to find a separating  $w$  in a **finite** number of updates.

#### Intuition:

The proof sketch (for conceptual understanding, not memorization) The proof shows that as the number of corrections  $k$  grows, the cosine of the angle  $\beta$  between  $w_t$  and the (unknown) ideal solution  $w^*$  satisfies  $\cos \beta \geq c\sqrt{k}$  for some constant  $c > 0$ . But  $\cos \beta$  can never exceed 1! So  $k$  must be bounded—the algorithm *cannot* keep making corrections forever. It must converge.

**Trick used:** negate every point in  $N$  (so  $-x \in P$  effectively), turning the two-class problem into a single-class problem: find  $w$  such that  $w^T p \geq 0$  for every  $p$  in the combined set  $P' = P \cup N^-$ .

## 4.3 Linearly Separable Sets: Formal Definition

### Definition

$P$  and  $N$  are **absolutely linearly separable** if there exist  $n + 1$  real numbers  $w_0, \dots, w_n$  such that:

$$\sum_i w_i x_i \geq w_0 \quad \forall (x_1, \dots, x_n) \in P$$

$$\sum_i w_i x_i < w_0 \quad \forall (x_1, \dots, x_n) \in N$$

## 5 Representation Power: One Perceptron is Not Enough

### 5.1 The XOR Problem

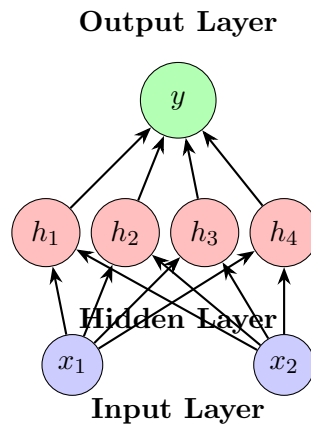
| $x_1$ | $x_2$ | XOR | Constraint                  |
|-------|-------|-----|-----------------------------|
| 0     | 0     | 0   | $w_0 + \sum w_i x_i < 0$    |
| 1     | 0     | 1   | $w_0 + \sum w_i x_i \geq 0$ |
| 0     | 1     | 1   | $w_0 + \sum w_i x_i \geq 0$ |
| 1     | 1     | 0   | $w_0 + \sum w_i x_i < 0$    |

These 4 constraints are mutually **contradictory** for a single linear boundary—no  $(w_0, w_1, w_2)$  can satisfy all four. XOR is fundamentally NOT linearly separable.

### 5.2 The Fix: Networks of Perceptrons

#### Intuition:

The key insight that revives neural networks One perceptron = one straight cut through space. But if you combine *several* perceptrons (each making its own cut) and then feed their outputs into *another* perceptron, you can carve out arbitrarily complex, non-linear regions. This is the birth of the **Multi-Layer Perceptron (MLP)**.



### Representation Power Theorem

**Any** boolean function of  $n$  inputs can be *exactly* represented by a network with:

- One hidden layer with (at most)  $2^n$  perceptrons
- One output perceptron

**Catch:** As  $n$  grows, the number of hidden perceptrons grows *exponentially*. This motivates **deep** (multi-layer) networks, which can represent complex functions far more *compactly* than a single wide hidden layer.

### *Intuition:*

Why does one perceptron per input combination work? The trick: design each hidden perceptron to fire for *exactly one* specific input combination and stay silent for all others (using a very negative bias like  $-2$  with  $\pm 1$  weights so only the exact matching input pushes the sum above threshold). Then the output layer just needs to know: “sum up (with the right output weight) whichever hidden neuron fired.” It’s essentially a lookup table implemented in weights!

## 6 Practice Problems

### Problem 1: MP Neuron Design [2 marks]

Design an MP neuron with 3 excitatory inputs that implements the **majority function** (output 1 if at least 2 of the 3 inputs are 1). What is  $\theta$ ?

### Problem 2: MP Neuron with Inhibition [2 marks]

Design an MP neuron with inputs  $x_1$  (excitatory),  $x_2$  (inhibitory) that computes  $x_1$  AND NOT  $x_2$ . Give  $\theta$  and explain.

### Problem 3: Linear Separability Check [2 marks]

Is the boolean function  $f(x_1, x_2, x_3) = 1$  iff exactly one of  $x_1, x_2, x_3$  is 1 (i.e., “exactly-one” function) linearly separable? Justify.

### Problem 4: Perceptron Update [3 marks]

Perceptron with  $w = [1, -1, 2]^T$  (including bias as  $w_0 = 1$ ). Point  $x = [1, 2, 1]^T$  (with  $x_0 = 1$ ) has true label 1 (in  $P$ ).

- (a) Compute  $w^T x$ . Is it correctly classified?
- (b) If misclassified, compute  $w_{\text{new}}$  after one perceptron update.

### Problem 5: Perceptron Update (Negative Class) [3 marks]

Perceptron with  $w = [0, 1, 1]^T$ . Point  $x = [1, 3, 2]^T$  has true label 0 (in  $N$ ).

- (a) Compute  $w^T x$ . Is it correctly classified?
- (b) If misclassified, compute  $w_{\text{new}}$ .

### Problem 6: Counting Boolean Functions [2 marks]

How many boolean functions exist for  $n = 3$  inputs? How many of the 2-input boolean functions are NOT linearly separable?

### Problem 7: XOR via Network [3 marks]

Using two hidden units  $h_1 = \text{AND}(x_1, \text{NOT } x_2)$  and  $h_2 = \text{AND}(\text{NOT } x_1, x_2)$ , and output  $y = \text{OR}(h_1, h_2)$ , verify this network computes XOR correctly for all 4 input combinations by filling a truth table.

### Problem 8: Geometric Perceptron [2 marks]

A perceptron has  $w = [2, 3]$  (no bias, i.e., decision boundary passes through origin). For point  $x = [3, -2]$ , is it classified as 1 or 0? What is the angle relationship (acute/obtuse) between  $w$  and  $x$ ?



## 7 Solutions

**Problem 1:** Majority of 3: need at least 2 of 3 inputs on.  $\boxed{\theta = 2}$ .  $g(x) = x_1 + x_2 + x_3 \geq 2 \Rightarrow y = 1$ .

**Problem 2:**  $x_1$  AND NOT  $x_2$  means: output 1 only when  $x_1 = 1, x_2 = 0$ . Use  $x_1$  excitatory,  $x_2$  inhibitory,  $\theta = 1$ . If  $x_2 = 1$  (inhibitory fires)  $\rightarrow$  output forced 0 regardless of  $x_1$ . If  $x_2 = 0$ :  $g = x_1 \geq 1$  only when  $x_1 = 1$ . Correct.

**Problem 3: Not linearly separable.** Truth table:  $(0,0,0) \rightarrow 0, (1,0,0) \rightarrow 1, (0,1,0) \rightarrow 1, (0,0,1) \rightarrow 1, (1,1,0) \rightarrow 0, (1,0,1) \rightarrow 0, (0,1,1) \rightarrow 0, (1,1,1) \rightarrow 0$ . Points like  $(1,0,0), (0,1,0), (0,0,1)$  (label 1) and  $(1,1,0), (1,0,1), (0,1,1)$  (label 0) cannot be separated by one hyperplane—similar to XOR generalized to 3D, this is the well-known “parity-like” function that fails linear separability.

**Problem 4:** (a)  $w^T x = 1(1) + (-1)(2) + 2(1) = 1 - 2 + 2 = 1 \geq 0$ . **Correctly classified** (predicted 1, true label 1). No update needed.

**Problem 5:** (a)  $w^T x = 0(1) + 1(3) + 1(2) = 5 \geq 0$ , so predicted 1, but true label is 0. **Misclassified.**

(b)  $w_{\text{new}} = w - x = [0, 1, 1]^T - [1, 3, 2]^T = [-1, -2, -1]^T$ .

**Problem 6:** For  $n = 3$ :  $2^{2^3} = 2^8 = \mathbf{256}$  functions. For  $n = 2$ :  $2^{2^2} = 16$  functions, exactly **2** (XOR and NXOR/XNOR) are not linearly separable.

**Problem 7:**

| $x_1$ | $x_2$ | $h_1$ | $h_2$ | $y = h_1 \text{ OR } h_2$ |
|-------|-------|-------|-------|---------------------------|
| 0     | 0     | 0     | 0     | 0                         |
| 1     | 0     | 1     | 0     | 1                         |
| 0     | 1     | 0     | 1     | 1                         |
| 1     | 1     | 0     | 0     | 0                         |

Matches XOR truth table exactly. **Verified.**

**Problem 8:**  $w^T x = 2(3) + 3(-2) = 6 - 6 = 0$ . On the boundary exactly (angle = 90). By convention  $y = 1$  if  $\geq 0$ , so **classified as 1**, with  $w \perp x$  (right angle, boundary case).